Lab3-Score Distillation Sampling Implementation

Yu-Hsuan Li (112550011)
NYCU
johnnyli122022@gmail.com

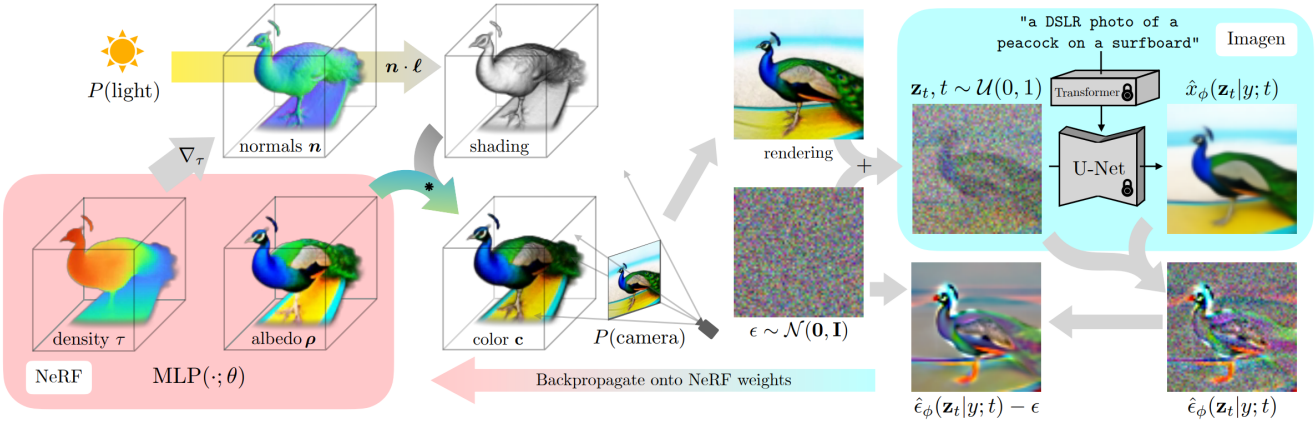


Figure 1. **Overview of score distillation.** A rendered image is noised and scored by a frozen text-to-image diffusion model, and the residual $\hat{\epsilon}_\phi - \epsilon$ is backpropagated to the 3D parameters. In this lab the 3D representation is replaced by a 2D latent. Figure from DreamFusion [5].

1. Introduction

Ordinary diffusion sampling starts from pure noise and runs the reverse process step by step to produce an image. Score distillation instead treats the image (here, a Stable Diffusion latent) as a set of free parameters and uses a frozen, pretrained diffusion model only as a *critic*: at every iteration the model looks at a noisy version of the current latent and tells us in which direction it should move to look more like the text prompt. This turns a diffusion model into a loss function, which is what makes text-to-3D methods such as DreamFusion [5] possible (Fig. 1).

In this assignment I implemented three score distillation methods on top of Stable Diffusion 2.1-base for text-to-image generation: Score Distillation Sampling (SDS) [5], Variational Score Distillation (VSD) [9], and Score Distillation via Inversion (SDI) [4]. Each method optimizes a $1 \times 4 \times 64 \times 64$ latent for the 10 evaluation prompts, and the results are evaluated with the CLIP ViT-B/32 image–text similarity [6]. I also ran the required ablations on the SDS guidance scale and on the SDI inversion steps and update interval, and an extra experiment with the original diffusion loss.

Section 2 describes the methods and my implementation, Sec. 3 reports the results and ablations, and Sec. 4 discusses why the plain diffusion loss fails and why VSD and SDI improve over SDS.

2. Methodology

Background. All three methods share the same frozen Stable Diffusion UNet $\epsilon_\theta(x_t, y, t)$ [7], which predicts the noise in a noisy latent x_t given a prompt y and timestep t. The forward process [2] produces $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$. Noise predictions use classifier-free guidance (CFG) [1], $\hat{\epsilon}_\theta = \epsilon_{\text{uncond}} + s\,(\epsilon_{\text{cond}} - \epsilon_{\text{uncond}})$, where the unconditional branch uses the negative prompt "low quality". The optimized parameter is the latent x_0, initialized from $\mathcal{N}(0, I)$ and decoded with the VAE only for visualization. The optimizer (AdamW, cosine schedule, gradient clipping) is the one provided in `main.py`.

2.1. Score Distillation Sampling (SDS)

Concept. SDS [5] noises the current latent with a random t and ϵ, asks the diffusion model to predict the noise, and uses the difference as the update direction: $\nabla_{x_0} L_{\text{SDS}} = w(t)\,(\hat{\epsilon}_\theta(x_t, y, t) - \epsilon)$. Intuitively, the residual $\hat{\epsilon}_\theta - \epsilon$ points from the current image towards what the model thinks a

clean image for this prompt looks like. This is the gradient of the ordinary diffusion loss with the UNet Jacobian term dropped; skipping it makes SDS cheap (no backpropagation through the UNet) and, as Sec. 4.1 shows, is also what makes it work at all.

Implementation. I sample $t \sim \mathcal{U}[20, 980]$ and ϵ, add noise with the scheduler, and predict the noise with CFG under `no_grad`. The gradient uses the DreamFusion weight $w(t) = 1 - \bar{\alpha}_t$. To feed this hand-made gradient to the optimizer, I write it as an MSE to a detached target $x_0 - \text{grad}$, whose derivative with respect to x_0 is exactly the SDS gradient.

```
def get_sds_loss(self, latents, text_embeddings,
     guidance_scale=7.5):
   B = latents.shape[0]
   t = torch.randint(self.min_step, self.max_step + 1, (
        B,), device=self.device)
   noise = torch.randn_like(latents)
   with torch.no_grad():
      latents_noisy = self.scheduler.add_noise(latents,
           noise, t)
      noise_pred = self.get_noise_preds(latents_noisy, t
           , text_embeddings, guidance_scale)
   w = (1 - self.alphas[t]).view(B, 1, 1, 1)
   grad = torch.nan_to_num(w * (noise_pred - noise))
   # d(loss)/d(latents) == grad (skips the UNet Jacobian
        )
   target = (latents - grad).detach()
   return 0.5 * F.mse_loss(latents, target, reduction="
        sum") / B
```

2.2. Variational Score Distillation (VSD)

Concept. In SDS the reference ϵ is pure random noise, so the update is noisy and SDS needs a very large guidance scale to converge, which causes over-smoothed and oversaturated results. VSD [9] replaces ϵ with the prediction of a second network ϵ_ϕ, a LoRA [3] on top of the same UNet, which is trained online to model the distribution of the images currently being generated. The update $w(t)\,(\hat{\epsilon}_\theta - \epsilon_\phi)$ then compares "what the pretrained model expects" with "what we currently generate", which is a much cleaner signal and works with a normal guidance scale of 7.5.

Implementation. The LoRA (rank 4 on the attention projections) is set up by the provided code. For ϵ_θ I temporarily disable the adapters and predict with CFG; for ϵ_ϕ I enable them and use only the conditional branch. Both predictions are made without gradients, and the VSD term uses the same detached-target trick as SDS, so it only updates the latent. The LoRA is trained with the standard denoising loss on a *detached* copy of the latent with a fresh t and ϵ, so it only updates the LoRA. The returned loss is the sum of both terms, and `main.py` steps the latent and the LoRA parameters together.

```
def get_vsd_loss(self, latents, text_embeddings,
     guidance_scale=7.5, lora_loss_weight=1.0):
   B = latents.shape[0]
   cond = text_embeddings.chunk(2)[1]
   t = torch.randint(self.min_step, self.max_step + 1, (
        B,), device=self.device)
   noise = torch.randn_like(latents)
   with torch.no_grad():
      latents_noisy = self.scheduler.add_noise(latents,
           noise, t)
      self.unet.disable_adapters() # eps_theta:
           pretrained UNet + CFG
      eps_pre = self.get_noise_preds(latents_noisy, t,
           text_embeddings, guidance_scale)
      self.unet.enable_adapters() # eps_phi: LoRA UNet,
           conditional only
      eps_lora = self.unet(latents_noisy, t,
           encoder_hidden_states=cond).sample
   w = (1 - self.alphas[t]).view(B, 1, 1, 1)
   grad = torch.nan_to_num(w * (eps_pre - eps_lora))
   target = (latents - grad).detach()
   loss_vsd = 0.5 * F.mse_loss(latents, target,
        reduction="sum") / B

   # LoRA learns the distribution of the current latents
   t2 = torch.randint(self.min_step, self.max_step + 1,
        (B,), device=self.device)
   noise2 = torch.randn_like(latents)
   noisy2 = self.scheduler.add_noise(latents.detach(),
        noise2, t2)
   pred2 = self.unet(noisy2, t2, encoder_hidden_states=
        cond).sample
   loss_lora = F.mse_loss(pred2.float(), noise2.float())
   return loss_vsd + lora_loss_weight * loss_lora
```

2.3. Score Distillation via Inversion (SDI)

Concept. SDI [4] keeps the SDS framework but fixes its noise: instead of a fresh random ϵ at every step, the noisy latent x_t is obtained by running DDIM [8] *backwards* from the current x_0. The noise is therefore consistent with the current image, so denoising x_t gives a target $\hat{x}_0$ that is a sharpened version of the current latent rather than an average over many random noises. The latent is then pulled towards $\hat{x}_0$ with a simple MSE. SDI also replaces the random timestep of SDS with a schedule that goes from large to small t, so the image first settles its layout and then refines its details.

Implementation. `invert_noise()` is a hand-written deterministic DDIM inversion ($\eta = 0$, as allowed): it splits $[0, t]$ into n steps and, at each step, predicts the noise with CFG scale -7.5, estimates $\hat{x}_0$, and re-noises it to the next (larger) timestep. In `get_sdi_loss()`, t is annealed linearly from 980 to 20 over the 1000 iterations. Every `update_interval` iterations the current latent is inverted to x_t, the noise is predicted with CFG 7.5, and a one-step denoise gives the target, which is cached. In between, the loss is only the MSE to the cached target, so most iterations need no UNet call at all.

```
@torch.no_grad()
def invert_noise(self, latents, target_t,
     text_embeddings, guidance_scale=-7.5, n_steps=10,
     eta=0.3):
   B = latents.shape[0]
   target = int(target_t.max().item())
   timesteps = torch.linspace(0, target, n_steps + 1,
        device=self.device).round().long()
   x = latents.clone()
   for t_cur, t_next in zip(timesteps[:-1], timesteps
        [1:]):
      a_cur, a_next = self.alphas[t_cur], self.alphas[
           t_next]
      eps = self.get_noise_preds(x, t_cur.expand(B),
           text_embeddings, guidance_scale)
```

Table 1. **CLIP score** (↑, ViT-B/32) per prompt. Best per row in bold.

Prompt	SDS	VSD	SDI
A red bus driving on a desert road	**0.3619**	0.3221	0.3608
a boat in a river	**0.3030**	0.2961	0.2922
A cabin surrounded by forests	0.3017	**0.3291**	0.3210
A church beside a lake	0.3243	**0.3295**	0.3103
A villa close to the pool	0.2833	0.2885	**0.2941**
A castle next to a river	0.2856	**0.2996**	0.2903
A burger on the table	**0.3035**	0.2715	0.2918
A dog sitting on grass	**0.2985**	0.2857	0.2651
a cat sitting on a table	**0.3267**	0.2837	0.3059
A car on the road	**0.2827**	0.2763	0.2791
Average	**0.3071**	0.2982	0.3010

```
        x0 = (x - (1 - a_cur).sqrt() * eps) / a_cur.sqrt()
        x = a_next.sqrt() * x0 + (1 - a_next).sqrt() * eps
    return x
```

```
def get_sdi_loss(self, latents, text_embeddings,
    guidance_scale=7.5,
            current_iter=0, total_iters=500,
                inversion_guidance_scale=-7.5,
            inversion_n_steps=10, inversion_eta=0.3,
                update_interval=25):
    B = latents.shape[0]
    progress = current_iter / max(1, total_iters) #
        anneal t: 980 -> 20
    t_val = int(round(self.max_step - (self.max_step -
        self.min_step) * progress))
    t = torch.full((B,), t_val, dtype=torch.long, device=
        self.device)

    if current_iter % update_interval == 0 or not hasattr
        (self, 'sdi_target'):
        with torch.no_grad():
            x_t = self.invert_noise(latents, t,
                text_embeddings, inversion_guidance_scale,
                            inversion_n_steps,
                                inversion_eta)
            eps = self.get_noise_preds(x_t, t,
                text_embeddings, guidance_scale)
            a_t = self.alphas[t].view(B, 1, 1, 1)
            target = (x_t - (1 - a_t).sqrt() * eps) / a_t.
                sqrt() # one-step denoise
            self.sdi_target = torch.nan_to_num(target).
                detach()
    return 0.5 * F.mse_loss(latents, self.sdi_target,
        reduction="sum") / B
```

3. Experiments & Results

All experiments use the 10 default prompts in `eval.sh` and the hyperparameters in Tab. 3, on a single RTX 4090. `main.py` does not fix a random seed, so re-running a setting gives slightly different images and scores.

3.1. Quantitative Results

All three methods are above the 0.28 threshold (Tab. 1). SDS has the highest average (0.3071), followed by SDI (0.3010) and VSD (0.2982), but the gap is small and the ranking changes from prompt to prompt: VSD wins the scenery prompts (cabin, church, castle), SDI wins the villa, and SDS wins most object-centric prompts.

Table 2. **Ablation study.** Average CLIP score (↑) over the 10 prompts. Time: average optimization time per prompt on one RTX 4090.

(a) SDS: `guidance_scale`					
Value	7.5	15	25	50	100
CLIP	0.2547	0.2837	0.3071	**0.3076**	0.3015
(b) SDI: `inversion_n_steps`					
Value	1	3	5	10	25
CLIP	0.2030	0.3020	0.3018	0.3010	**0.3054**
Time (s)	13.4	21.2	26.2	43.7	93.3
(c) SDI: `sdi_update_interval`					
Value	25	75	200	500	1000
CLIP	**0.3010**	0.2963	0.2960	0.2942	0.2226
Time (s)	43.7	16.8	8.2	6.4	5.4

The scores do not fully agree with my visual impression. CLIP ViT-B/32 sees a 224×224 image and mostly checks whether the prompt's object is present and recognizable; it is not very sensitive to blur. A blurry but clean, centered object from SDS therefore scores well, while the busy high-frequency textures of VSD and the occasional odd layout of SDI (*e.g.* the striped road for the car prompt) cost them points even when the image looks sharper.

3.2. Qualitative Results

Figure 2 shows clear differences between the methods. **SDS** produces the right content and layout, but every image is blurry, low in detail and has washed-out colors. **VSD** is much sharper and has rich detail (the dog's fur, the castle, the cabin), but the colors are over-saturated and the backgrounds often break into painterly, mosaic-like textures (the rock wall behind the bus, the water around the boat, the road). **SDI** gives the cleanest results overall: sharp objects, natural colors, and simple backgrounds (the villa, burger, bus and boat). Its failure cases are a somewhat flat, illustration-like style and, occasionally, a repeated pattern instead of a single object (the car prompt).

3.3. Ablation Study

Each ablation changes one hyperparameter and keeps all others as in Tab. 3. Every value is run on all 10 prompts; Tab. 2 reports the average CLIP score, and Figs. 3 to 5 show three of the prompts. For SDI I also report the average optimization time per prompt.

3.3.1. SDS: `guidance_scale`

The guidance scale has a large effect on SDS (Fig. 3, Tab. 2(a)). At 7.5 and 15 the images are blurry blobs with almost no detail, and for some prompts the object does not appear at all. Increasing the scale makes the images sharper and more saturated; 25–50 gives the best scores (0.307–0.308). At 100 the colors become too strong and noisy textures appear, so the score drops slightly. SDS therefore



Figure 2. **Qualitative comparison** of SDS, VSD and SDI on the 10 evaluation prompts.



Figure 3. **Ablation on SDS `guidance_scale`.**



Figure 4. **Ablation on SDI `inversion_n_steps`.**

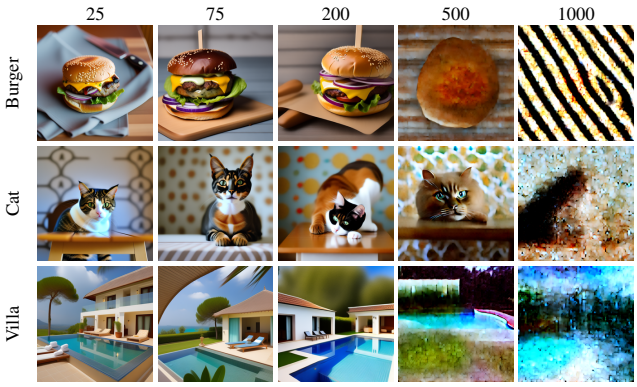


Figure 5. **Ablation on SDI `sdi_update_interval`.**

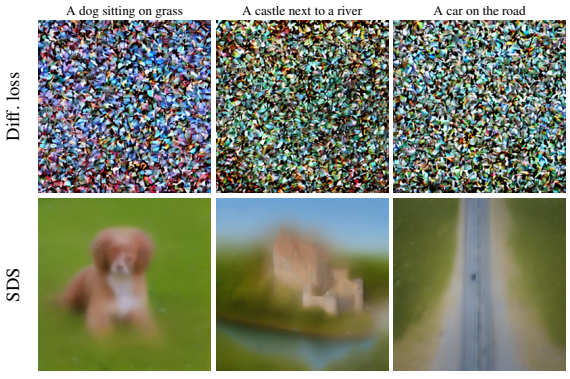


Figure 6. **Original diffusion loss** (top) vs. SDS (bottom).

Table 3. **Hyperparameters** for each task.

Hyperparameter	SDS	VSD	SDI
`guidance_scale`	25	7.5	7.5
`lr`	0.01	0.01	0.005
`steps`	500	500	1000
`lora_lr`	–	1e-4	–
`lora_rank`	–	4	–
`lora_loss_weight`	–	1.0	–
`inversion_n_steps`	–	–	10
`inversion_guidance_scale`	–	–	-7.5
`inversion_eta`	–	–	0.3
`sdi_update_interval`	–	–	25

needs a much larger scale than the usual 7.5 for ordinary sampling: because the random noise changes at every step, the guidance signal is weak on average and has to be amplified to be noticed.

3.3.2. SDI: `inversion_n_steps`

With a single inversion step the method fails completely (0.2030, nearly blank images in Fig. 4): one large DDIM step from 0 to t is too inaccurate, so the inverted noise does not match the image and the target is meaningless. From 3 steps on, the inversion is good enough and the score is stable around 0.30; 25 steps gives the best score (0.3054) and slightly more detailed images. The runtime grows almost linearly with the number of steps (13 s for 1 step, 44 s for 10, 93 s for 25), so there is a clear trade-off; the default of 10 steps is a reasonable middle ground.

3.3.3. SDI: `sdi_update_interval`

Refreshing the target more often gives better results (Fig. 5, Tab. 2(c)). Between 25 and 500 the score decreases slowly (0.301 to 0.294) and the images become less refined, because the latent keeps chasing an older target that was computed when the image and the timestep were different. At the extreme of 1000, the target is computed only once at the first iteration, from a random latent at $t \approx 980$, so the result is just a blurry color blob (0.2226). A larger interval is much cheaper (5 s instead of 44 s per prompt), but the default of 25 gives the best quality.

3.4. Hyperparameters

I used the default hyperparameters from the README and `main.py` for all three tasks, listed in Tab. 3. All other settings are shared: negative prompt "low quality", timestep range $[0.02, 0.98]$, AdamW with a cosine schedule and 100 warm-up steps, gradient clipping at norm 1, and fp32. Since all three methods already passed the CLIP threshold with these values, I did not tune them further.

4. Discussion

4.1. Why the Original Diffusion Loss Fails

To produce Fig. 6, I used the same setup as SDS (CFG 25, lr 0.01, 500 steps) but minimized $\mathcal{L}_{\text{Diff}} = \|\hat{\epsilon}_\theta(x_t, y, t) - \epsilon\|^2$ directly, backpropagating through the UNet (separate script `scripts/diff_loss.py`). The results contain no recognizable object, only colorful high-frequency noise, while SDS with the same settings produces the expected content.

The difference is only one term. The full gradient is $(\hat{\epsilon}_\theta - \epsilon)\,\frac{\partial \hat{\epsilon}_\theta}{\partial x_t}\,\frac{\partial x_t}{\partial x_0}$, and SDS simply drops the UNet Jacobian in the middle. Intuitively, the diffusion loss does not ask the image to look real; it asks the UNet to predict the noise well. Through the Jacobian, the optimizer finds that tiny high-frequency changes to the input are the fastest way to reduce the UNet's error, much like an adversarial attack, so the latent drifts towards patterns that are meaningless to humans. Without the Jacobian, the residual $\hat{\epsilon}_\theta - \epsilon$ itself points towards a cleaner, prompt-consistent image, which is why SDS works.

4.2. Why SDI and VSD Outperform SDS

SDS. The typical SDS artifacts in my results are blur, missing detail, and washed-out colors at normal guidance, and over-saturation at very high guidance (Figs. 2 and 3). The cause is the random noise: every step uses a new ϵ and t, so each step pushes the image in a slightly different direction, and over many steps these directions average out into a smooth "average" image. A large guidance scale is needed to keep a strong enough signal, but it also exaggerates colors.

VSD. VSD replaces the random noise with a learned prediction of the current images. The random part that SDS suffers from is now present on both sides of the difference and largely cancels, leaving a cleaner direction from "what I generate now" towards "what the pretrained model expects". This is why VSD works with a normal guidance of 7.5 and gives much sharper details than SDS in Fig. 2. In my runs it also introduced over-saturated textures, which lowered its CLIP score; I believe a better-tuned LoRA (*e.g.* learning rate) would reduce them.

SDI. SDI keeps the SDS update but chooses the noise by DDIM inversion instead of at random. Because the noise is consistent with the current image, the target from one-step denoising is a sharper version of that image instead of a blurry average, and the latent converges smoothly towards it; the large-to-small timestep schedule further separates layout from detail. This gives the cleanest images in Fig. 2 at guidance 7.5. The inversion ablation supports this: with a poor inversion (1 step) SDI collapses, while any reasonably accurate inversion gives good results.

Although SDS has the highest average CLIP score in my runs, the differences are small, and visually both VSD and

SDI are clearly sharper and more detailed, which CLIP ViT-B/32 does not fully reflect.

5. Conclusion

I implemented SDS, VSD and SDI for text-to-image generation with Stable Diffusion 2.1-base, including a hand-written DDIM inversion for SDI. All three methods pass the CLIP threshold (SDS 0.3071, SDI 0.3010, VSD 0.2982). Visually, SDS is blurry and washed out, VSD is sharp but over-saturated, and SDI gives the cleanest images. The ablations show that SDS needs a much larger guidance scale than ordinary sampling, that SDI needs an accurate inversion (at least a few steps), and that its target must be refreshed often enough. Directly optimizing the diffusion loss produces only noise, which shows why SDS drops the UNet Jacobian. Finally, VSD and SDI improve over SDS for the same reason: they replace the random noise in the SDS update with something that is consistent with the current image, either a learned noise prediction or an inverted one, which gives a cleaner and more stable optimization signal.

References

[1] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022. 1

[2] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Adv. Neural Inform. Process. Syst.*, 2020. 1

[3] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *Int. Conf. Learn. Represent.*, 2022. 2

[4] Artem Lukoianov, Haitz Sáez de Ocáriz Borde, Kristjan Greenewald, Vitor Campagnolo Guizilini, Timur Bagautdinov, Vincent Sitzmann, and Justin Solomon. Score distillation via reparametrized DDIM. In *Adv. Neural Inform. Process. Syst.*, 2024. 1, 2

[5] Ben Poole, Ajay Jain, Jonathan T. Barron, and Ben Mildenhall. Dreamfusion: Text-to-3d using 2d diffusion. In *Int. Conf. Learn. Represent.*, 2023. 1

[6] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *Int. Conf. Mach. Learn.*, 2021. 1

[7] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *IEEE Conf. Comput. Vis. Pattern Recog.*, 2022. 1

[8] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *Int. Conf. Learn. Represent.*, 2021. 2

[9] Zhengyi Wang, Cheng Lu, Yikai Wang, Fan Bao, Chongxuan Li, Hang Su, and Jun Zhu. Prolificdreamer: High-fidelity and diverse text-to-3d generation with variational score distillation. In *Adv. Neural Inform. Process. Syst.*, 2023. 1, 2